M-ADSAI-ZM002-26

Reachy Mini: Hardware & Technical Capability Assessment

Caio De Alcantara Santos (2604500), Cas Limpens (2400410), Donné Deguelle (2301667) and Gustavo Gonçalves Da Costa (2604824).

Contents

Introduction 3
Purpose 3
Intended Use Case 3
Scope 4
Research Questions 5
Hardware 6
Computing Power and AI models 8
Local vs. External Processing 8
Requirements of AI Models 9
Expected Limitations 10
External Processing Alternative 11
Other models 12
Selected AI models for benchmarking 16
Performance Evaluation and Deployment 17
Connectivity & Networking 20
Software & Available Functions 21
Robot Software / SDK 21
Available APIs 21
Hardware Access 21
Available AI Integration 21
Conclusion & Recommendation 22
Bibliography 23

Introduction

Purpose

The purpose of this research is to investigate the hardware and technical capabilities of the Reachy Mini and determine whether the robot is suitable for our intended application. The research focuses on the processing capabilities of the robot itself, the available sensors and interaction hardware, the available software interfaces, and the connectivity options required to operate the robot. A specific focus of this research is determining whether AI-based pose estimation and movement-recognition models can be executed directly on the Reachy Mini's internal hardware. If the available hardware is insufficient for the required AI processing, this research will investigate the feasibility of using an external computer or other hardware for computationally intensive tasks. The results of this research will allow the team to make an informed decision about which functionality can be performed directly on the Reachy Mini and which functionality may require additional hardware or infrastructure.

Intended Use Case

The intended use case is to use the Reachy Mini as an interactive and motivating device for patients performing physiotherapy exercises at home. The robot could support the patient by providing encouragement, feedback and interactive responses during exercises. It may also react to the patient's movements and use its physical movements, audio output and other interaction capabilities to make the exercises more engaging. AI-based pose estimation and movement recognition are expected to play an important role in this use case. The system should be able to identify which movement the patient is performing, determine whether the movement is performed correctly and, in a later stage, provide a score or other form of feedback.

Scope

The following areas will be investigated:

- The Reachy Mini's internal hardware and specifications.
- Camera capabilities and software accessibility.
- Microphone, speaker and audio capabilities.
- Motors, movement capabilities and control interfaces.
- Other relevant sensors.
- Operating system, SDK and available APIs.
- Processing capabilities for computer vision and AI.
- Feasibility of running pose-estimation and movement-recognition models locally.
- Options for external AI processing.
- Networking and connectivity options.
- Network requirements for the intended use case.
- Additional equipment required for testing.
- Technical limitations and potential risks.

The research does not aim to develop the final application. Instead, it focuses on establishing whether the Reachy Mini provides the technical capabilities and infrastructure required for further development.

Research Questions

The following research questions define the focus of this research.

Main Question

- *To what extent can the Reachy Mini detect a person, recognize physiotherapy-related exercises, and provide expressive feedback while operating within its computational constraints?*

Subquestions

1. *What hardware and technical specifications does the Reachy Mini have?*
2. *What functions and interfaces are available for the camera, audio, sensors and robot movement?*
3. *What software, SDKs, APIs and libraries are available for developing applications for the Reachy Mini?*
4. *What computing capabilities does the Reachy Mini have for running computer vision and AI models locally?*
5. *Can suitable pose-estimation and movement-recognition models run locally on the Reachy Mini, and what limitations should be expected?*
6. *If local processing is insufficient, how feasible is it to use an external computer for AI processing?*
7. *What connectivity and network setup is required, and is Wi-Fi sufficient for our intended use case?*
8. *What additional hardware or infrastructure is required to reliably develop and test the intended system?*

Hardware

Main Computer

- **CPU**: Broadcom BCM2711 quad-core Cortex-A72 at 1.5 Ghz (Raspberry Pi Ltd, 2026).
- **GPU**: Broadcom VideoCore VI at 500 MHz (Broadcom BCM2711, n.d.).
- **RAM**: 4 Gigabytes of LPDDR4 (Hugging Face, n.d.) (Raspberry Pi Ltd, 2026).
- **Storage**: 16 Gigabytes of eMMC flash storage (Hugging Face, n.d.) (Raspberry Pi Ltd, 2026).
- **Operating system**: Custom version of Raspberry Pi OS (Reachy Mini OS) (Pollen Robotics, n.d.).
- **Architecture:** ARM v8 (Raspberry Pi Ltd, 2026).

Camera

- **Camera type**: Raspberry pi camera v3 wide angle (Sony IMX708) (Hugging Face, n.d.).
- **Resolution**: 12 Megapixel (Hugging Face, n.d.).
- **FPS**: 50 FPS video capture (Hugging Face, n.d.).
- **Field of view**: 120 degrees (Hugging Face, n.d.).
- **Is it suitable for pose estimation?**: At the time of writing, we were able to test the Reachy Mini face tracking app. Which did well. There are other people that have used the Raspberry Pi Camera Module v3 for pose detection (Alif, 2023). The camera is the same as on the Reachy Mini.

Audio

- **Microphones**: 4 times the PDM MEMS digital mic (Hugging Face, n.d.).
- **Speaker**: 5W 4Ω speaker (Hugging Face, n.d.).
- **Voice interaction possibility?** We have been able to interact via voice with the Reachy Mini. By using the Reachy Mini conversation app. The Reachy Mini was also able to respond.

Motors & Movement

- **Number of motors**: 9 motors, 2 antenna motors (Dynamixel XL330-M077-T), 6 steward motors (Dynamixel XC330-M288-T) and 1 body turning motor (Dynamixel XC330-M288-PG) (Hugging Face, n.d.).
- **movement**: Figure 1 shows that the head of the Reachy Mini can move upwards and downwards (z-axis), sideways (y-axis) and forwards as well as backwards (x-axis). The antennas on the robot's head are also able to rotate and the robot can rotate its body.

Figure 2 shows that the heads x-axis has a minimum of -1.5cm and a maximum of +2.5cm. The y-axis has a minimum of -4cm and a +4cm maximum. For the z-axis is the minimum -4cm and the maximum +2.5cm. The head roll has a -40 degrees minimum and +40 degrees maximum. The head pitch has the same maximum and minimum as the head roll. The robot body has a -155 degrees minimum and +155 degrees maximum. Both the antennas can be moved 180 degrees both ways (Hugging Face, n.d.).

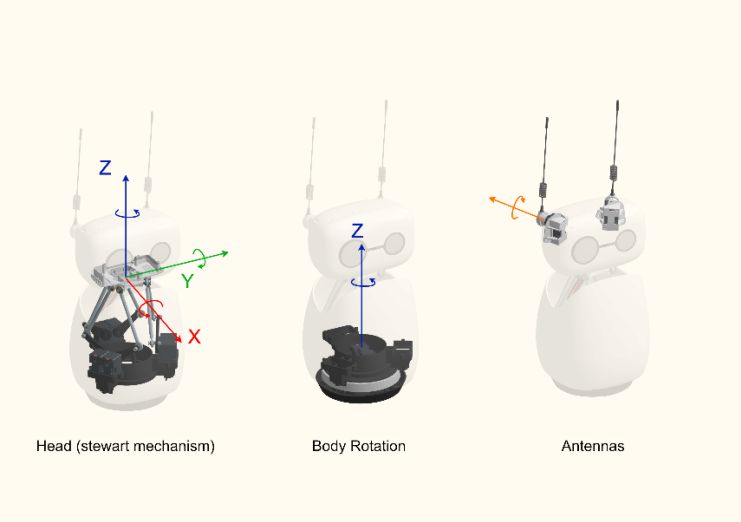


Figure 1: motor movement axis of the Reachy Mini: image by Pollen Robotics

Nominal Position

Axis	Min	Max
Tx (head)	-1.5cm	+2.5cm
Ty (head)	-4cm	+4cm
Tz (head)	-4cm	+2.5cm
Rx (head roll)	-40°	+40°
Ry(head pitch)	-40°	+40°
Rz (head yaw)	-60°	+60°
Rz (body yaw)	-155°	+155°
R Right antenna	-180°	+180°
R Left antenna	-180°	+180°

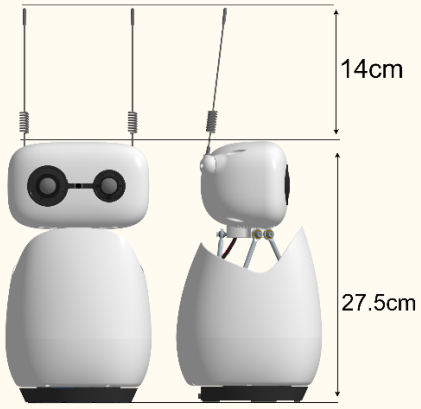


Figure 2: minimum and maximum movement Reachy Mini: image by Pollen Robotics

Other Sensors

- **IMU**: the wireless of the Reachy Mini we use for this project has an inertial measurement unit (Pollen Robotics, n.d.).
- **Position**: All 9 motors have a position sensor (Robotis, n.d.), (Robotis, n.d.), (Robotis, n.d.).

Computing Power and AI models

Local vs. External Processing

There are two possible approaches for processing AI models with the Reachy Mini: local processing and external processing.

With local processing, the AI model runs directly on the Reachy Mini's internal hardware. The Reachy Mini Wireless uses a Raspberry Pi Compute Module 4 with 4 GB of RAM and 16 GB of flash storage (Hugging Face, n.d.). This approach has the advantage that the camera data can be processed directly on the robot, without having to continuously send it to another computer. This can reduce network dependency and communication latency. Another advantage of local AI is that the data does not have to be sent to an external AI service. Local AI generally means that inference is performed on the user's own hardware rather than on a remote server (Local LLM Team, sd). For our application, this could be beneficial because the camera captures a patient's movements.

However, local processing is limited by the hardware available in the Reachy Mini. The robot's CM4 is primarily intended to operate the robot and its applications and does not provide the computing power of a dedicated modern GPU system. Therefore, more demanding AI models may not achieve sufficient performance when running directly on the robot. With external processing, the Reachy Mini can capture the camera data and send it to another computer. The external computer performs the AI inference and sends the resulting pose or movement information back to the Reachy Mini. This allows us to use significantly more powerful hardware for AI processing. The disadvantage is that the system becomes dependent on the network connection and additional latency is introduced. Pollen Robotics has tested the Reachy Mini and an external laptop on the same 5 GHz local network and measured approximately 100 ms total glass-to-glass latency in their media setup (Pollen Robotics, sd). They also note that the actual latency depends on the robot hardware, network and client hardware.

For our project, the most suitable approach therefore depends on whether the selected pose-estimation model can achieve sufficient performance on the Reachy Mini itself.

Requirements of AI Models

The AI component of our application can be divided into two different tasks:

1. Pose estimation: determining the position of a person's body joints.
2. Activity or movement recognition: determining which movement the person is performing and potentially whether it is performed correctly.

ADSAI has already been experimenting with some models:

Model	Type	Description	Computational demand
BlazePose	2D	Fast 2D human pose detection	Relatively low
ViTPose	2D	Modern transformer-based 2D pose estimation	Medium–high
WHAM	3D	3D human motion/pose estimation	High
CameraHMR	3D	Recent 3D human pose and shape estimation	Very high

Tabel 1: Known ADSAI AI models, by Maarten Vaessen

BlazePose is interesting for initial testing because lightweight pose-estimation pipelines can run on Raspberry Pi hardware. Google provides an official MediaPipe Pose Landmarker example specifically designed to run on a Raspberry Pi and perform real-time pose landmark detection from camera images (Pose Landmarker, sd). ViTPose is more computationally demanding. Implementations are available for CPU and GPU inference, but some implementations report real-time performance above 30 FPS specifically on modern NVIDIA GPUs and Apple Silicon (Vitpose, sd). WHAM is a 3D human motion estimation model that reconstructs human motion from video. Compared with BlazePose and ViTPose, which focus on 2D pose estimation, WHAM provides 3D motion information. This makes it relevant for more detailed movement analysis, but also more computationally demanding than the 2D approaches (WHAM, sd). CameraHMR is even more demanding. Its implementation uses PyTorch and CUDA 11.8 in the documented setup, while the project is designed for detailed 3D human pose and shape estimation (CameraHMR, sd). This makes it considerably less suitable as a first candidate for local execution on the Reachy Mini.

But, an important limitation is that none of these four models directly solve the activity-recognition part of our application. BlazePose, ViTPose, WHAM and CameraHMR primarily provide information about the person's pose or motion. A separate activity-recognition or movement-classification step would therefore likely be required. This distinction is important because our intended application requires the system to eventually identify movements and determine whether they are performed correctly.

Expected Limitations

Several factors can limit the ability to run these models directly on the Reachy Mini.

Processing power

- The Reachy Mini Wireless has a Raspberry Pi Compute Module 4 with 4 GB RAM and 16 GB storage (Hugging Face, n.d.). This is significantly more constrained than the hardware commonly used for computationally intensive computer-vision models.

Memory

- Only 4 GB of RAM is available. The operating system, Reachy Mini software, camera processing and AI model would all compete for the available memory.

Model complexity

- The four candidate models have very different computational requirements. A lightweight 2D model such as BlazePose is much more realistic for local testing than a complex 3D human mesh/pose model such as CameraHMR.

Real-time requirements

- Our application is intended to react to a patient's movements. Therefore, simply being able to execute a model is not enough. We need to determine whether it can run with an acceptable frame rate, inference time and response time. For example, a model that processes one frame every several seconds may technically run on the robot but would not be suitable for real-time interaction.

Competing processes

- The Reachy Mini also needs to operate its robot functions, camera, audio and other software. Running an AI model simultaneously may reduce the resources available for the rest of the system.

Network latency

- If processing is moved to an external computer, the network introduces additional latency.

External Processing Alternative

If the Reachy Mini cannot process the selected AI model sufficiently fast, the AI processing can be moved to an external computer. The Reachy Mini supports communication with external computers through its software interfaces, for example GStreamer (GStreamer Installation, sd). The official API provides network connectivity to the robot, while the SDK supports access to robot control and media functionality. In terms of network architecture, the Reachy Mini and external computer could be connected to the same access point, with the access point providing a connection to Eduroam and the internet. You can also use network segmentation to isolate the robot and related devices from the rest of the network so the External Computer can only send and receive data from the Reachy Mini, while the Reachy Mini itself can connect trough Eduroam to the Internet.

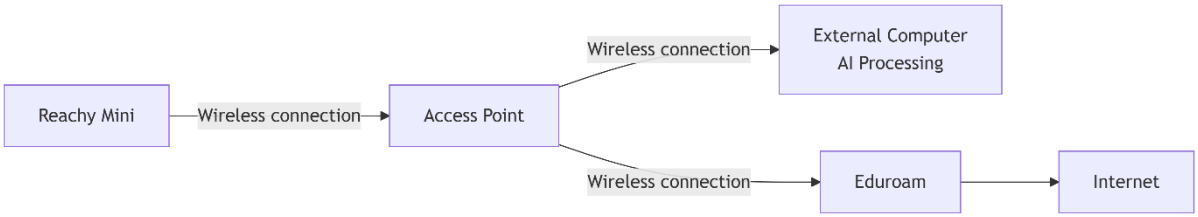


Figure 3: Reachy Mini with external AI computing

This approach would allow us to use more computationally demanding without being restricted to the processing capabilities of the Reachy Mini. The main trade-off is that the system becomes dependent on the external computer and the wireless network connection. The external computer must also be sufficiently powerful to run the selected model in real time. For the first prototype, this architecture is worth testing alongside local processing rather than immediately choosing one approach.

Other models

TSM (TSM-MobileNetV2):

Temporal Shift Module (TSM) is a lightweight mechanism designed to enable efficient temporal modeling. Temporal modeling is the process of using past frames to enhance the understanding of events in a sequence (ScienceDirect, n.d.). In video understanding using 2D convolutional neural network (CNNs), without adding additional computation. Instead of using heavy 3D convolutions for temporal modeling. TSM shifts output from a CNN layer across video frames. This shift allows each frame to receive information from neighboring frames, simulating the effect of 3D convolutions but at the computational cost of a 2D CNN. TSM shifts only a fraction of the output, otherwise data movement would be heavily increased (Lin, Gan, & Han, TSM: Temporal Shift Module for Efficient Video Understanding, 2019).

TSM is licensed under the MIT license. The Github repository contains mostly python code but also provides examples to test and train the model. Figure 4 shows how the model is able to detect the two fingers zoom signal (Lin, Gan, & Han, TSM: Temporal Shift Module for Efficient Video Understanding, 2024).



Figure 4: TSM detecting two fingers zoom: image by (Lin, Gan, & Han, TSM: Temporal Shift Module for Efficient Video Understanding, 2024)

MoViNets (MoViNet-A0):

MoViNets or Mobile Video Networks is a family of efficient video networks process video frames as they arrive. It uses stream buffering technics to process video in not overlapping parts of a fixed length. MoViNets uses temporal ensembling. Which is a technique that recovers some accuracy that is lost when using stream buffers. Without adding any inference cost. Two streaming MoViNets are trained independently with identical architectures but at half the frame rate, keeping the same total duration and thus halving each model's input frames. At inference, the same video is fed into both networks, with one network's frames offset by a single frame

relative to the other. The output from the models is averaged to provide a final prediction (Kondratyuk, et al., 2021).

TensorFlow shows how MoViNet can be setup. Which requires ffmpeg, mediapy, opencv, tensorflow, Tqdm, medipy, numpy matplotlib and PIL. In Figure 5 the MoViNet detects jumping jacks. The model gets more confident in its detection over time (Tensorflow, 2024).

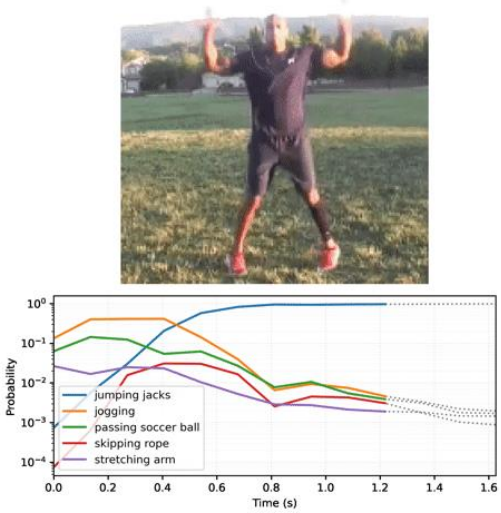


Figure 5: MoViNet jumping jacks detection: image by TensorFlow.

GraphEnet:

GraphEnet uses a graph neural network (GNN). The GNN is made of two components. The initial graph is built from event stream and it uses a method named SCARF (Set of Centre Active Receptive Fields) in parallel to the GNN to process the events to usable data. The model looks at parts of the image and draws a straight line through the moving pixels. If a line isn't clear enough, it is ignored. This drawing process runs in parallel with the GNN. This allows the model to handle large amounts of datapoints every second (Goyal, Thuong, Glover, Mizuno, & Bartolozzi, GraphEnet: Event-driven Human Pose Estimation with a Graph Neural Network, 2025).

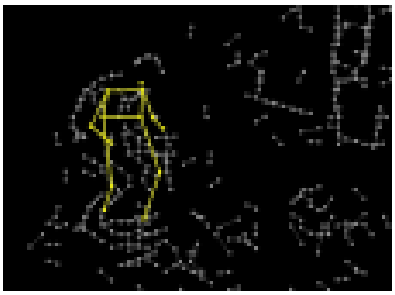


Figure 6: Result GaphEnet: image by (Goyal, Thuong, Glover, Mizuno, & Bartolozzi, GraphEnet: Event-driven Human Pose Estimation with a Graph Neural Network, 2025).

As mentioned GraphEnet uses a graph neural network. However, as visible in Figure 6 it is still represented in skeleton form in the final output.

The model is available on Github under the Attribution-NonCommercial-ShareAlike 4.0 International license. Which allows its use and modification if the following conditions are met: the (original) authors must be properly credited, the derivative work is under the same license and it is not used for commercial purposes. The code is written in python and requires additional libraries like pytorch-geometric as provided in the requirements.txt (Goyal, Thuong, Glover, Mizuno, & Bartolozzi, GraphEnet-NeVi-ICCV2025, 2025).

FSPose

FSPose is a diverse framework designed to balance accuracy and computational needs in human pose estimation for videos. FSPose adapts its architecture based on the type of frame and does not use the same network for all frames like other models. The framework divides video frames into Keyframes and non-keyframes. For keyframes it uses the HRNet W32 neural network. Which has higher accuracy and computational needs than the neural network used for the non-keyframes. Which is the MobileNetV2 which has lower accuracy, but also lower computational needs. When it has processed a keyframe, it will process the following non-keyframes with the lighter neural network. By splitting the frames this way over different neural networks, lowers the average load (Xu, Komorita, & Kawamura, FSPose: A Heterogeneous Framework with Fast and Slow Networks for Human Pose Estimation in Videos, 2023).

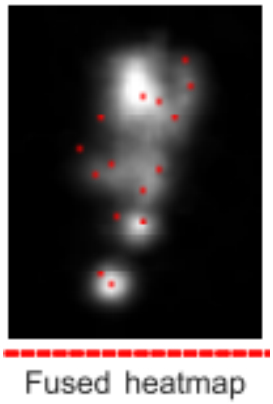


Figure 7: Result FSPose: image by (Xu, Komorita, & Kawamura, FSPose: A Heterogeneous Framework with Fast and Slow Networks for Human Pose Estimation in Videos, 2023).

Figure 7 is an example of the end result provided by FSPose. Which creates a combination of a heatmap with dots.

FSPose is written primarily in python and the Github repository includes instructions to install the model via docker. But also contains the code, including requirements.txt. From the repository we can also conclude that the authors used Cuda to run the model. It is unclear if Cuda is required to run the model. The model is licensed under the Apache license version 2.0 (Xu, Komorita, & Kawamura, FSPose: A Heterogeneous Framework with Fast and Slow Networks for Human Pose Estimation in Videos, 2023).

PySlowFast:

PySlowFast is an Apache 2.0 licensed video understanding codebase from Facebook AI Research (FAIR) that provides video classification models. The repository includes python implementations of multiple methods. The goal of PySlowFast is to provide a high-performance pytorch codebase. Which provides video backbones for video understanding research on different tasks. PySlowFast includes implementations of the following backbone architectures: SlowFast, Slow, Non-local Network, C2D, MViTv1/MViTv2, Rev-ViT/Rev-MViT, I3D and X3D. Figure 8 shows a picture of the SlowFast demo. Which clearly shows the ability of the model to detect what the people are doing (Meta, 2026).

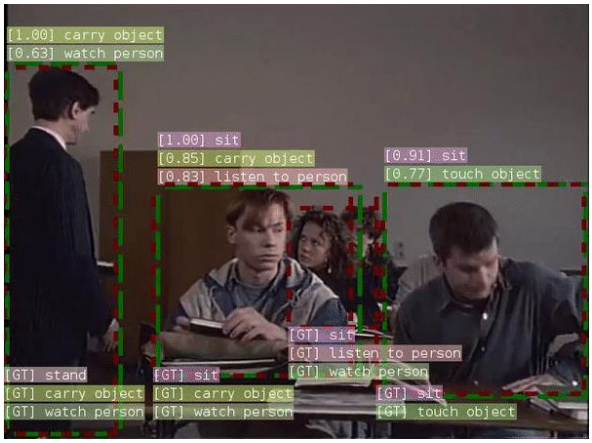


Figure 8: SlowFast demo: image by Meta.

SAM2 (Segment Anything Model 2):

SAM2 (Segment Anything Model 2) is a unified foundation model designed for promptable visual segmentation in both images and videos. SAM2 processes videos frame-by-frame using a streaming architecture equipped with a memory bank. The memory bank stores spatial information and object pointers from previous frames. It maintains a FIFO queue of the most recent frames and prompted frames. Which allows the model to remember object's appearance and trajectory. Once prompted the model predicts the segmentation mask for that frame and propagates it to subsequent frames. If the model loses the object, the user can provide a new prompt in a later frame (Ravi, et al., n.d.). SAM2 is however not feasible to run on the Reachy Mini, because it requires CUDA (Gallagher, 2024).

SAM2 is licensed under the Apache license version 2.0 but also contains cc-torch which is licensed under the 3-clause BSD license. The code is written in python, primarily using Jupyter notebooks. Figure 9 shows how SAM2 is able to detect humans and vehicles (Meta, 2026).

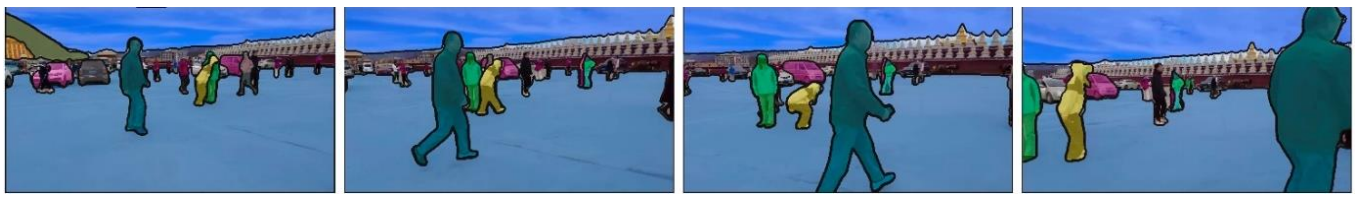


Figure 9: SAM2 detecting vehicles and people: image by Meta.

Selected AI models for benchmarking

Figure 11 shows the benchmark results for: BlazePose, MoViNet-A0, TSM-MobileNetV2 and ViTPose. These benchmarks were run on the laptop of Cas, which has a more powerful CPU and dedicated GPU compared to the Reachy Mini. Also, the RAM of the Reachy Mini is 4 GB, which is lower than the RAM of the Laptop. The benchmark was conducted using a 10 second video, without pose estimation. In the benchmark on the Laptop, the BlazePose model outperforms MoViNet-A0, TSM-MobileNetV2 and ViTPose in speed. Though the BlazePose model was not tested with GPU acceleration in our implementation, it still outperforms the other models which are able to use the GPU. In this benchmark, BlazePose has an average latency of 18.90 ms, a P95 latency of 20.69 ms and an FPS of 52.92.

Model	Status	Avg ms	P95	FPS	Device
BlazePose	OK	44.19	71.56	19.82	CPU
MoViNet-A0	OK	49.99	93.82	17.45	cuda
TSM-MobileNetV2	OK	45.52	76.17	18.71	cuda
ViTPose	OK	51.54	64.61	17.59	cuda

Figure 10: Laptop benchmark status

Benchmark	Model	Avg. Latency (ms)	P95 (ms)	FPS	RAM / VRAM (MB)
CPU	BlazePose	18,90	20,69	52,92	178,1
CPU	MoViNet-A0	36,05	45,25	27,74	114,0
CPU	TSM-MobileNetV2	139,28	185,11	7,18	178,6
CPU	ViTPose	197,80	258,12	5,06	70,1
GPU	MoViNet-A0	37,03	54,45	27,01	19,8
GPU	TSM-MobileNetV2	36,51	41,27	27,39	103,0
GPU	ViTPose	48,92	61,54	20,44	350,5

Figure 11: Laptop benchmark results

FINAL REACHY MINI RESULTS

Model	Status	Avg ms	P95	FPS	Camera
BlazePose	OK	217.22	251.59	3.92	3.77
MoViNet-A0	OK	529.44	604.71	1.77	1.77
TSM-MobileNetV2	OK	2053.89	2130.01	0.46	0.49
ViTPose	OK	5829.31	5906.60	0.17	0.17

Figure 12: Benchmark results Reachy Mini

Figure 12 shows the benchmark results on the Reachy Mini. BlazePose performs the best of the tested models, with 3.92 FPS. MoViNet-A0 has a 55% worse FPS rate than BlazePose. TSM-MobileNetV2 has a approximately 88% worse FPS rate and ViTPose has an approximately 96% worse FPS rate than BlazePose. We were able to access the camera on the Reachy Mini directly, which removes the 10 fps cap. Using OpenCV we were able to get approximately 12 fps while

using MoveNet Lightning INT8. Which is a 25% improvement over the previous 10 fps. We were also able to mount an external ssd using the usb-c port on the robot.

We have tried to build models for the Reachy Mini's GPU using OpenGL and Vulkan. We were unable to do this. Because we were unable to disable the compilation with the AES instruction set. We cannot use the robots GPU. So, we tried to run the pose detection on an laptops GPU instead with the camara stream from the Reachy Mini. We tested this and compared this to the newest CPU implementation we made.

Performance Evaluation and Deployment

1. Performance results

Two configurations were tested for the real-time activity recognition system: local processing on the Reachy Mini and remote pose estimation on an Ubuntu laptop equipped with an NVIDIA GeForce GTX 1650 GPU.

Metric	**Local processing on Reachy Mini**	**Remote processing on laptop**
Processing hardware	Reachy Mini CPU	NVIDIA GTX 1650 GPU
Inference runtime	LiteRT / TensorFlow Lite	ONNX Runtime with CUDA
Camera input resolution	192 × 108 pixels	192 × 108 pixels
Measured processing speed	23.80 FPS average	Typically 30–31 FPS
Mean pose inference time	34.32 ms	Approximately 5.1 ms in earlier measurements
Network required for inference	No	Yes
Session and robot control	Local	Remains on the Reachy Mini

Table 2: Comparison local Reachy Minin processing and remote laptop processing.

The local benchmark processed 4,721 frames over 198.40 seconds, achieving an average of 23.80 FPS. Pose inference took an average of 34.32 ms per frame, with a 95th-percentile inference time of 83.62 ms. The session update logic took an average of 2.46 ms.

The laptop implementation typically processed 30–31 frames per second, with occasional drops to between 16 and 28 FPS. ONNX Runtime confirmed that the CUDA execution provider was active, allowing the laptop's GPU to accelerate pose estimation.

The laptop therefore achieved a higher frame rate, but the difference in overall processing speed was relatively small considering the substantial difference in computing power between the two devices.

2. Investigating the frame-rate limitation

The results suggest that the laptop's GPU is not necessarily the limiting factor in the remote configuration. The camera is configured to provide a 192 × 108 pixel MJPEG stream, and the current camera configuration uses a default frame duration of approximately 33.3 ms, corresponding to roughly 30 FPS.

The Reachy Mini's IMX708 camera supports higher frame rates in certain sensor modes. For example, some cropped modes can operate at around 120 FPS, while a full-field-of-view mode can reach approximately 56 FPS. These are sensor-mode limits rather than guaranteed application frame rates.

It would therefore be possible to investigate higher frame rates by changing the camera configuration and adjusting the server's capture loop. However, the actual capture rate would need to be measured independently before concluding that the camera is the only bottleneck. JPEG encoding, network transmission, frame decoding and other application overhead may also affect the remote pipeline.

More importantly, the current results do not indicate that higher FPS is necessary for a satisfactory user experience. Both configurations already operate at real-time speeds.

3. Activity recognition and user experience

The exercise-session tests demonstrated that the system can provide an interactive, guided workout experience. The Reachy Mini speaks to the user, introduces exercises, demonstrates movements using its head and antennas, tracks the user's body and provides feedback during the session.

The local benchmark successfully counted all four repetitions in the squat and side-bend rounds. The laptop implementation generally maintained a higher frame rate, but this does not automatically translate into better activity recognition.

4. Preferred deployment approach

Although the laptop has significantly more computing power, the Reachy Mini achieved an average of 23.80 FPS while running MoveNet locally on its CPU. This is already a strong result for an embedded device and is sufficient to support responsive exercise interactions. Running the complete application locally on the Reachy Mini is therefore the preferred deployment approach, provided that activity recognition and repetition counting are sufficiently reliable. Local processing reduces dependence on an external laptop and network connection, simplifies deployment and keeps the system self-contained. Remote inference remains useful during development, experimentation and model optimization. It provides additional computing headroom and allows alternative models to be evaluated without being constrained by the robot's CPU.

5. Conclusion Evaluation and Deployment Considerations

The performance evaluation shows that local inference on the Reachy Mini achieves 23.80 FPS on average which is more than enough for activity recognition.

Higher camera frame rates could be investigated by modifying the camera configuration and capture pipeline, but this should be treated as an optimization rather than a requirement. The current frame rates are already suitable for real-time interaction.

Ultimately, the goal is not to maximize FPS, but to provide reliable movement recognition and a smooth, engaging exercise experience.

Connectivity & Networking

Available Connections

- **Wi-Fi**: Wi-Fi 5 with 2.4Ghz and 5Ghz dual-band (Hugging Face, n.d.).
- **Bluetooth**: Version 5.0 with Bluetooth low energy (BLE) (Raspberry Pi Ltd, 2026).
- **USB**: The Reachy Mini wireless has a USB type-c port for data transfer (Hugging Face, n.d.).
- **Ethernet?:** The Reachy Mini does not have an Ethernet port, even do the main board has a chip to support Gigabyte ethernet (Raspberry Pi Ltd, 2026) (Hugging Face, n.d.).
- **Other interfaces?**: the reachy mini also has a charge port. It cannot be charged with a USB type-c cable (Hugging Face, n.d.).

How Does Reachy Connect?

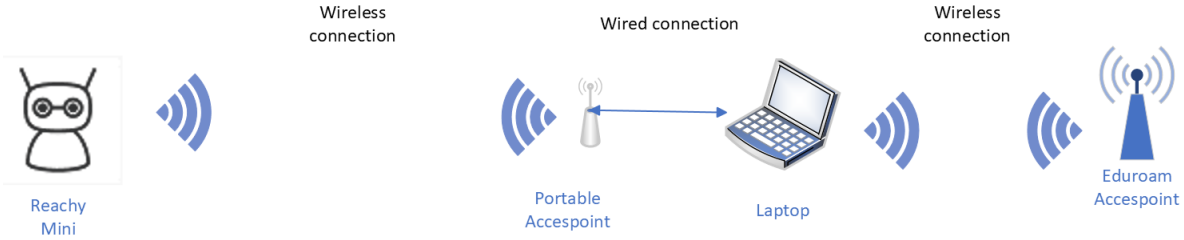


Diagram 3: Reachy Mini to Eduroam network

In diagram 1 we show how the Reachy Mini can connect to the internet. A portable accesspoint is used to share the laptops Eduroam connection with the Reachy Mini. The wireless accespoint creates its own network which allows us to connect to the Reachy Mini with a stable connection.

Is Wi-Fi Enough?

- **bandwidth**: The WI-FI bandwidth for the 5Ghz band is up to 433mbps (Infineon, n.d.) (Raspberry Pi Ltd, 2026). If the camera streams its footage to a laptop this is not expected to consume more than 24 mbps of bandwidth (Hakopyan, 2025). Which means that there is still enough bandwidth for the other functionality that is needed for the project.
- **Reliability**: We have tested the Reachy Mini while connected to the portable accespoint. From when we started using this portable accespoint the connection has been stable while testing apps like the Face-tracking app.

Software & Available Functions

Robot Software / SDK

There are two SDKs available for the Reachy Mini. One python SDK and one JavaScript SDK. According to the documentation both SDKs provide full control of the robot (Hugging Face, n.d.). The Reachy Mini python SDK is supported on Windows, MacOS and Linux. The python SDK requires python version 3.10-3.12 and the latest version of git, including git LFS. The reachy-mini python package needs to be installed into a virtual environment which can then be used to program the robot (Huggingface, n.d.). The Reachy Mini ships a JavaScript browser SDK over WebRTC. The npm package pollen-robotics/reachy-mini-sdk exposes: the ReachyMini class, the host shell and embed adapter. The documentation for the Reachy Mini JavaScript SDK also shows that WebRTC, audio data and video data are peer to peer with the robot and the controlling device. This is not the case for the http signals, they go through a signaling server and then to the robot (Hugging Face, n.d.).

Available APIs

The Swager UI of the Reachy Mini is reachable from: http://reachy-mini.local:8000/docs. In the Swager UI all available API options are listed. Through the API we are able to Install, remove and manage apps. We are also able to get a Unified robot description (URDF) through the API, move the robot into different positions, play sounds and more. (Hugging face, n.d.)

Hardware Access

The hardware can be accessed using the python SDK with the following functions. We can record audio with the Reachy Mini by using the function: mini.media.start_recording() and mini.media.get_audio_sample() to save the audio sample. To play recorded audio on the Reachy Mini the mini.media.push_audio_sample() function can be used. The frames from the camera can be accessed using mini.media.get_frame() when the media backend is set. The Reachy Mini can be moved using the mini.goto_target() function. In this function will then be specified what part the Reachy Mini needs to move and how long that will take. So for example, the body can be turned 20 degrees using body_yaw=np.deg2rad(20) and can the antennas be moved 50 degrees using antennas=np.deg2rad([50, 50]) (Hugging Face, n.d.).

Available AI Integration

The Reachy Mini supports head tracking by using the mini.start_head_tracking(). Function (Hugging Face, n.d.). While this is not sufficient for full pose estimation it might be useful to detect the user before the session has started.

Conclusion & Recommendation

The main question: to what extent can the Reachy Mini detect a person, recognize physiotherapy-related exercises, and provide expressive feedback within its computational constraints? Can be answered positively: all are possible on the robot itself.

The Reachy Mini Wireless is built around a Raspberry Pi Compute Module 4 , with a 12 MP wide-angle IMX708 camera, four microphones, a 5 W speaker, an IMU and nine Dynamixel motors. All hardware is accessible via the Python SDK, the JavaScript SDK and the REST API, covering camera frames, audio recording and playback, motor control and built-in head tracking. The software ecosystem is therefore fully sufficient for the intended application.

Computationally, the robot is constrained but still usable. The GPU of the Pi Compute Module 4 proved unusable so AI must run on the CPU. Heavy models are unsuitable locally. The lightweight model MoveNet achieved 23.80 FPS locally, which is sufficient for real-time interaction. Direct camera access removed the default 10 FPS cap.

Crucially, pose estimation alone does not classify exercises. A separate recognition step is required, and tests showed this works: the local pipeline successfully counted all repetitions during squat and side-bend rounds while the robot guided the session with speech, head movements and antenna movement.

External processing is feasible but unnecessary for deployment. A laptop with a GTX 1650 reached 30-31 FPS remotely, yet the gain was marginal because computing power is not the bottleneck. Wi-Fi is adequate: the stream uses well under 24 Mbps of the available 433 Mbps, and connections were stable. A portable access point sharing an Eduroam connection solves campus network restrictions. Required additional equipment is limited to a development laptop, a portable access point.

Recommendations

- Deploy locally. The 23.80 FPS local pipeline is sufficient and keeps the robot self-contained. Which avoids dependence on a laptop in a patient's home
- Use MoveNet as the local pose estimator. Which provided the best performance on the Reachy Mini’s CPU.
- Focus follow-up research on CMAS exercises and clinical robustness. The technical feasibility is established. The remaining questions are functional. Follow-up research should evaluate the system specifically against the CMAS exercise set, testing how reliably it recognizes correct versus incorrect execution of each CMAS exercise across different patients, body types and lighting conditions.

Bibliography

Alif, M. (2023, November 9). *Setting Up Pose-detection AI with Raspberry Pi 4 + Coral USB Accelerator*. Retrieved from medium.com: https://medium.com/@Mujadded/setting-up-pose-detection-ai-with-raspberry-pi-4-coral-usb-accelerator-b29a38d5d780

Broadcom BCM2711. (n.d.). Retrieved from wikidevi.wi-cat.ru: https://wikidevi.wi-cat.ru/Broadcom_BCM2711

CameraHMR. (n.d.). Retrieved from Github: https://github.com/udaya67/CameraHMR-New

Gallagher, J. (2024, July 30). *What is Segment Anything 2 (SAM 2)?* Retrieved from blog.roboflow.com/: https://blog.roboflow.com/what-is-segment-anything-2/

Goyal, G., Thuong, P. C., Glover, A., Mizuno, M., & Bartolozzi, C. (2025, October 9). *GraphEnet: Event-driven Human Pose Estimation with a Graph Neural Network*. Retrieved from arxiv.org: https://arxiv.org/pdf/2510.07990

Goyal, G., Thuong, P. C., Glover, A., Mizuno, M., & Bartolozzi, C. (2025, October 14). *GraphEnet-NeVi-ICCV2025*. Retrieved from github.com: https://github.com/event-driven-robotics/GraphEnet-NeVi-ICCV2025?tab=readme-ov-file

GStreamer Installation. (n.d.). Retrieved from Huggingface: https://huggingface.co/docs/reachy_mini/v1.10.0/en/SDK/gstreamer-installation

Hakopyan, M. (2025, June 13). *IP Camera Bandwidth Planning Guide for Small Offices*. Retrieved from asi-networks.com: https://www.asi-networks.com/blog/ip-camera-bandwidth-planning-guide-smbs/

Hugging Face. (n.d.). *JavaScript SDK runtime reference*. Retrieved from huggingface.co: https://huggingface.co/docs/reachy_mini/SDK/javascript-sdk

Hugging Face. (n.d.). *Python SDK Reference*. Retrieved from huggingface.co: https://huggingface.co/docs/reachy_mini/SDK/python-sdk

Hugging Face. (n.d.). *Reachy Mini Hardware Datasheet*. Retrieved from huggingface.co: https://huggingface.co/docs/reachy_mini/platforms/reachy_mini/hardware

Hugging face. (n.d.). *REST API*. Retrieved from huggingface.co: https://huggingface.co/docs/reachy_mini/API/rest-api

Huggingface. (n.d.). *Installation Guide*. Retrieved from huggingface.co: https://huggingface.co/docs/reachy_mini/SDK/installation?install-uv=Windows&install-git=Windows&install-method=%F0%9F%93%A6+Option+A%3A+PyPI

Infineon. (n.d.). *CYW43455*. Retrieved from https://www.infineon.com/: https://www.infineon.com/part/CYW43455

Kondratyuk, D., Yuan, L., Li, Y., Zhang, L., Tan, M., Brown, M., & Gong, B. (2021, April 18). *MoViNets: Mobile Video Networks for Efficient Video Recognition*. Retrieved from arxiv.org: https://arxiv.org/pdf/2103.11511

Lin, J., Gan, C., & Han, S. (2019, August 22). *TSM: Temporal Shift Module for Efficient Video Understanding*. Retrieved from arxiv.org: https://arxiv.org/pdf/1811.08383

Lin, J., Gan, C., & Han, S. (2024, July 11). *TSM: Temporal Shift Module for Efficient Video Understanding*. Retrieved from github.com: https://github.com/mit-han-lab/temporal-shift-module

Local LLM Team. (n.d.). *Local LLM*. Retrieved from https://www.local-llm.net/learn/what-is-local-ai/

Meta. (2026, March 16). *PySlowFast*. Retrieved from github.com: https://github.com/facebookresearch/slowfast

Meta. (2026, December 16). *SAM 2: Segment Anything in Images and Videos*. Retrieved from github.com: https://github.com/facebookresearch/sam2

Pollen Robotics. (n.d.). Retrieved from https://pollen-robotics.com/reachy-mini/blog/media-stack/

Pollen Robotics. (n.d.). *Reachy Mini ISO for Wireless*. Retrieved from github.com: https://github.com/pollen-robotics/reachy-mini-os

Pollen Robotics. (n.d.). *Reachy Mini Wireless*. Retrieved from store.pollen-robotics.com: https://store.pollen-robotics.com/products/reachy-mini-wireless-version

Pose Landmarker. (n.d.). Retrieved from Github: https://github.com/google-ai-edge/mediapipe-samples/blob/main/examples/pose_landmarker/raspberry_pi/README.md

Raspberry Pi Ltd. (2026, June 30). *Raspberry Pi Compute Module 4*. Retrieved from pip-assets.raspberrypi.com: https://pip-assets.raspberrypi.com/categories/634-raspberry-pi-compute-module-4/documents/RP-008168-DS-4-cm4-datasheet.pdf

Ravi, N., Gabeur, V., Hu, Y.-T., Hu, R., Ryali, C., Ma, T., . . . Feichtenhofer, C. (n.d.). *SAM 2: Segment Anything in Images and Videos*. Retrieved from scontent-ams2-1.xx.fbcdn.net: https://scontent-ams2-1.xx.fbcdn.net/v/t39.2365-6/464917098_581932941165933_4465312900778079623_n.pdf?_nc_cat=105&ccb=1-7&_nc_sid=3c67a6&_nc_ohc=JeKm8NtQl-AQ7kNvwF4c39M&_nc_oc=Adrew8_hNHAT1X0eLasY7Ib1JVXtnAputntQvgy6ZduvaJboaSvX7EEHNonfeODkhkK02nEtCo2xyQi

Robotis. (n.d.). *XC330-M288-T*. Retrieved from docs.robotis.com: https://docs.robotis.com/docs/dxl/model_reference/x_series/xc_series/xc330-m288/

Robotis. (n.d.). *XL330-M077-T*. Retrieved from docs.robotis.com: https://docs.robotis.com/docs/dxl/model_reference/x_series/xl_series/xl330-m077/

Robotis. (n.d.). *XL330-M288-T*. Retrieved from docs.robotis.com: https://docs.robotis.com/docs/dxl/model_reference/x_series/xl_series/xl330-m288/

ScienceDirect. (n.d.). *Temporal Modeling*. Retrieved from www.sciencedirect.com: https://www.sciencedirect.com/topics/computer-science/temporal-modeling

Tensorflow. (2024, March 9). *MoViNet for streaming action recognition*. Retrieved from tensorflow.org: https://www.tensorflow.org/hub/tutorials/movinet

Vitpose. (n.d.). Retrieved from Github: https://github.com/lum1t4/vitpose

WHAM. (n.d.). Retrieved from Github: https://github.com/yohanshin/WHAM

Xu, J., Komorita, S., & Kawamura, K. (2023, June 6). *FSPose: A Heterogeneous Framework with Fast and Slow Networks for Human Pose Estimation in Videos*. Retrieved from jstage.jst.go.jp: https://www.jstage.jst.go.jp/article/transinf/E106.D/6/E106.D_2022EDP7182/_pdf

Xu, J., Komorita, S., & Kawamura, K. (2023, March 16). *FSPose: A Heterogeneous Framework with Fast and Slow Networks for Human Pose Estimation in Videos*. Retrieved from github.com: https://github.com/Fenax79/fspose